

DISTRIBUTED COMPUTING PRACTICALS

Python Programs – Practical 1, 2 & 3

Practical 1: Concurrent Echo Client-Server Application

server.py

```
import socket
import threading

HOST = "127.0.0.1"
PORT = 5000

def handle_client(client_socket, address):
    print("Client connected:", address)

    while True:
        data = client_socket.recv(1024).decode()

        if not data:
            break

        print("Client:", data)
        client_socket.send(("Echo: " + data).encode())

    client_socket.close()
    print("Client disconnected:", address)

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(5)

print("Echo Server started...")
print("Waiting for clients...")

while True:
    client_socket, address = server.accept()

    thread = threading.Thread(
        target=handle_client,
        args=(client_socket, address)
    )

    thread.start()
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

print("Connected to Echo Server")
print("Type 'exit' to close")

while True:
    message = input("You: ")

    if message.lower() == "exit":
        break

    client.send(message.encode())

    response = client.recv(1024).decode()
    print("Server:", response)
```

```
client.close()
```

Run: Open two terminals. Run *python server.py* in the first and *python client.py* in the second.

Practical 2: Swapping Variable Values Using IPC Through Thread Programming

```
import threading
import queue

q1 = queue.Queue()
q2 = queue.Queue()

def thread1():
    a = 10
    b = 20

    print("Before Swapping:")
    print("A =", a)
    print("B =", b)

    # Send values using IPC
    q1.put(b)
    q2.put(a)

def thread2():
    new_a = q1.get()
    new_b = q2.get()

    print("\nAfter Swapping:")
    print("A =", new_a)
    print("B =", new_b)

t1 = threading.Thread(target=thread1)
t2 = threading.Thread(target=thread2)

t1.start()
t2.start()

t1.join()
t2.join()

print("\nProgram Completed")
```

Sample Output

```
Before Swapping:
A = 10
B = 20

After Swapping:
A = 20
B = 10

Program Completed
```

Practical 3: Implement Remote Procedure Call (RPC) Chat Server

rpc_server.py

```
from xmlrpc.server import SimpleXMLRPCServer

def send_message(name, message):
    return name + " says: " + message

def welcome(name):
    return "Welcome " + name + "!"

server = SimpleXMLRPCServer(("127.0.0.1", 8000))

server.register_function(send_message, "send_message")
server.register_function(welcome, "welcome")

print("RPC Chat Server started...")
print("Waiting for client...")

server.serve_forever()
```

rpc_client.py

```
import xmlrpc.client

server = xmlrpc.client.ServerProxy("http://127.0.0.1:8000")

name = input("Enter your name: ")

print(server.welcome(name))

while True:
    message = input("You: ")

    if message.lower() == "exit":
        break

    response = server.send_message(name, message)

    print("Server:", response)
```

Run: Open two terminals. Run *python rpc_server.py* first, then *python rpc_client.py*.

Example: Pritesh → Hello → Server: Pritesh says: Hello